

Anatomy of an HTML document

The last lesson left you with a small example file and the idea that HTML describes structure and meaning. This lesson takes that file apart and explains the pieces that show up in almost every page. By the end, you should be able to read, write, and modify the basic skeleton of an HTML document.

Every modern HTML document follows the same basic shape:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My first page</title>
  </head>
  <body>
    <h1>Hello, web</h1>
    <p>This is a paragraph of text.</p>
  </body>
</html>
```

Each line has a job:

- `<!DOCTYPE html>` tells the browser to use modern HTML rules. It is not really a tag; it is a declaration. Without it, the browser may fall back to old compatibility behavior called quirks mode.
- `<html lang="en">` is the root element of the page. The `lang` attribute tells screen readers, search engines, and translation tools what language the content is in.
- `<head>` holds information *about* the page: its title, its character encoding, links to CSS, scripts, and so on. Users do not see this content directly.
- `<body>` holds everything users actually see and interact with: headings, paragraphs, images, forms, buttons.

The `head` and `body` are siblings inside `html` . Their jobs are different, and you should write both explicitly in normal HTML pages.

Tags, elements, and attributes

Almost everything inside an HTML document is an element. A typical element looks like this:

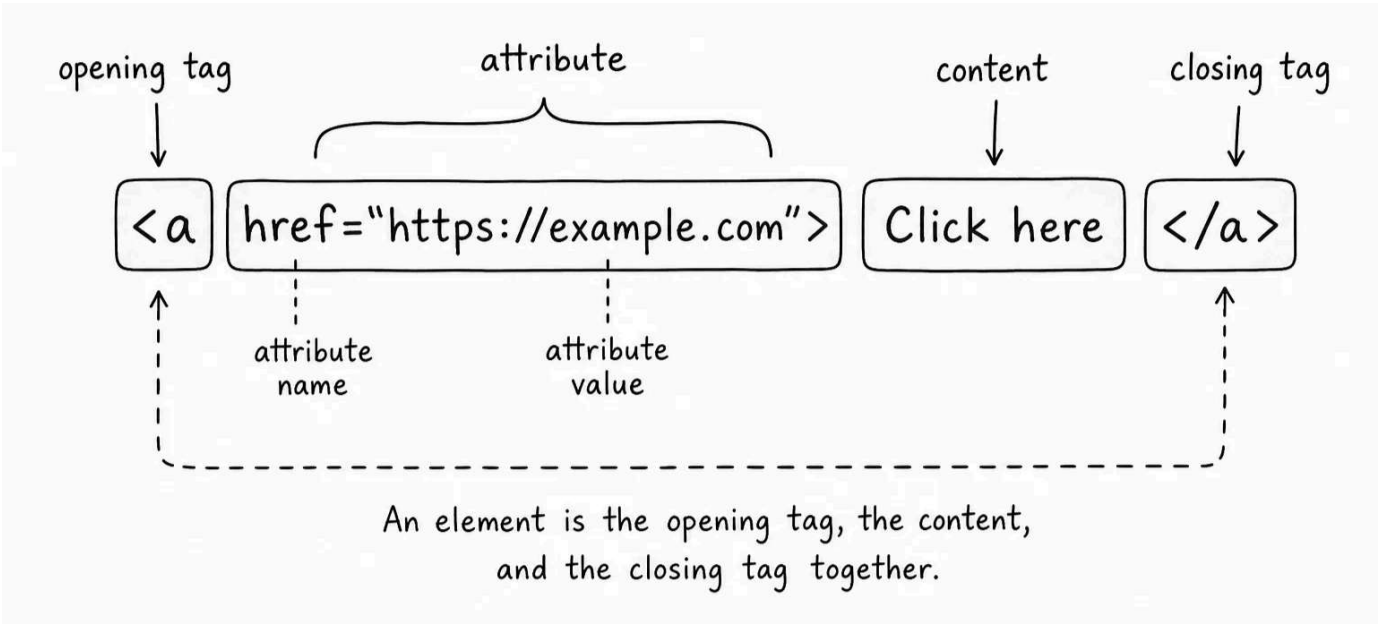
html

Click here

There are four parts to notice:

- `<a>` is the **opening tag**. It marks the start of the element.
- `</a>` is the **closing tag**. The slash means "closing".
- `Click here` is the **content** of the element.
- `href="https://example.com"` is an **attribute**, a name/value pair inside the opening tag that gives the element extra information.

Together, the opening tag, the content, and the closing tag form one **element**. The tag is just the marker; the element is the whole thing.



Most attributes are written as `name="value"`. Single quotes work too, but double quotes are the convention. A few attributes are *boolean*: writing the name on its own is enough. For example, the `disabled` attribute on a button:

html

<button disabled>Submit</button>

You can stack many attributes on the same element:

html

<input type="email" name="email" required placeholder="you@example.com">

Three attributes that show up everywhere

Three attributes appear in almost every HTML file you will read or write: `id`, `class`, and `data-*`.

The `id` attribute is a unique label for one element on the page. Only one element should have a given id. CSS, JavaScript, and fragment links all use ids to find a specific element.

html

```
<h2 id="pricing">Pricing</h2>
```

In that snippet, the `h2` has `id="pricing"` . A URL like `https://roadmap.sh#pricing` opens `https://roadmap.sh` and scrolls to the element with that id.

The `class` attribute is a shared label. Many elements can have the same class, and one element can have many classes separated by spaces. In the example below, the paragraph has two classes: `lead` and `intro` .

html

```
<p class="lead intro">A short summary paragraph.</p>
```

Classes are mostly used by CSS to apply styles, and by JavaScript to find groups of elements.

The `data-*` attributes are for your own custom data. Any attribute starting with `data-` is treated as valid, and the browser will not complain about it.

html

```
<button data-product-id="42">Add to cart</button>
```

Browsers ignore `data-*` attributes by default. JavaScript can read them, and CSS can select elements by them when needed.

💡 `id` and `class` are your main hooks into the page. You will use them in every CSS and JavaScript lesson in the future.

Void elements and nesting

Not every element wraps content. Some elements are written as a single tag and never have a closing tag. These are called **void elements**.

html

```

<br>
<hr>
<input type="text" name="email">
<meta charset="UTF-8">
<link rel="stylesheet" href="style.css">
```

You do not write `</img>` or `</br>` . They have no content to wrap, so a closing tag would have nothing to close. You may see older examples written like `<br />` , but in HTML, `<br>` is the normal form.

Elements that *do* have content can contain other elements. This is called **nesting**. The rule is simple: elements must close in the order they opened, like balanced brackets.

This is correct:

html

```
<p>This text is <strong>important</strong> and clear.</p>
```

This is broken. The **strong** element should close before the **p** :

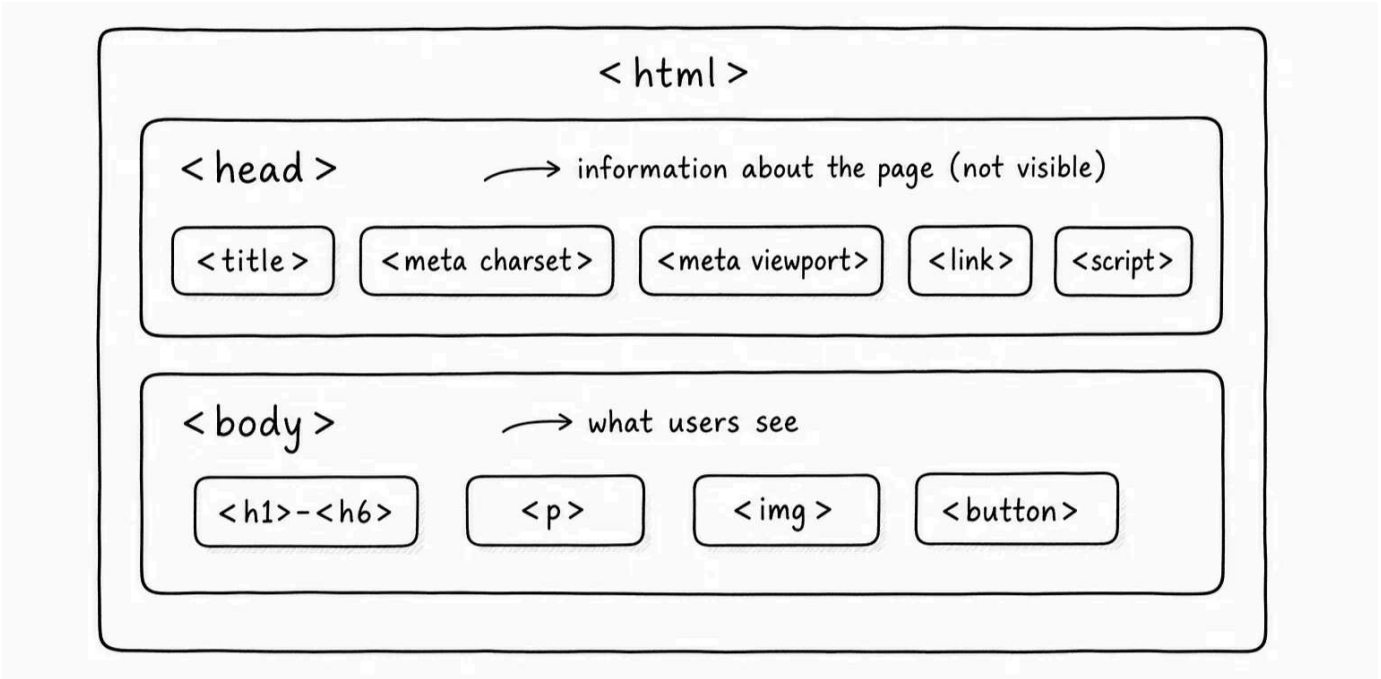
html

```
<p>This text is <strong>important</p></strong>
```

Browsers try to fix broken nesting on their own, but the result is not reliable. Keep your tags balanced.

What goes in the head

The **head** is for information that helps the browser interpret and present the page. It is not where you put the main content users read, click, or type into.



The most important things in the head are these five.

<title> sets the text shown in the browser tab and in search results.

html

```
<title>Pricing - Acme</title>
```

<meta charset="UTF-8"> tells the browser how to decode the text in the file. UTF-8 covers virtually every character used by humans today.

 **Always set UTF-8**

Without `charset="UTF-8"` , names like `André` or `Müller` may appear as garbled characters in your users' browsers. Modern frameworks set this for you, but it is worth knowing why the line is there.

`<meta name="viewport" content="width=device-width, initial-scale=1">` tells mobile browsers to use the actual device width when rendering. Without it, phones may try to render the page as if it were on a wide desktop and zoom out.

`<link rel="stylesheet" href="style.css">` loads a CSS file.

The `rel="stylesheet"` attribute tells the browser what kind of resource the link is pointing to.

`<script src="app.js" defer></script>` loads a JavaScript file.

The `defer` attribute tells the browser to keep parsing the HTML and run the script after the document is built. Without `defer` , the browser stops parsing to download and run the script, which slows the page down.

ⓘ Don't worry about about the `link` and `script` tags yet, we will cover them in detail when we learn about CSS and JavaScript.

That is most of what beginners need from the head. Other tags belong there too, such as social previews, favicons, and analytics, but you can pick those up as you need them.

Comments and entities

Comments are a way to add information to your HTML files that is ignored by browsers. Comments can span multiple lines; you start a comment by writing `<!--` and end with `-->` - everything between these two is ignored by the browser.

html

```
<!-- This is a comment. The browser ignores it. -->

<!--
  This is a multiline. You can
  break it down in multiple lines
  to make it easier for you to read.
-->
```

Comments are useful for leaving notes for yourself or other developers, or for temporarily hiding a block of HTML while you work on it.

HTML also has **entities** for characters that would otherwise be interpreted as HTML, or that are easier to write by name. The most common are:

- `<` for `<`
- `>` for `>`
- `&` for `&`
- `©` for ©

You mostly need entities when you want to display a character that would otherwise look like markup. To show the text `<div>` on the page, write `<div>`. Otherwise the browser thinks you are starting a `div` element.

Have a look at the example below to get an idea:

html

```
<!-- in the example below, browser will try to parse <meta> tag -->
<p>You can use <meta> tag to tell the browser more about the page</p>

<!-- this will be fine -->
<p>You can use &lt;meta&gt; tag to tell the browser more about the page</p>
```

HTML in templates and frameworks

Hand-writing every HTML file is rare in real projects. Many apps generate HTML through templates, components, or server frameworks. A template usually looks like normal HTML with a few placeholders for data:

html

```
<title>{{ pageTitle }} - Acme</title>
```

When the server renders the page, it fills in `{{ pageTitle }}` with real data. The resulting HTML is what the browser receives.

This is why the document skeleton still matters even when a framework writes most of the HTML for you. A wrong `charset` can make user names show up as garbage. A missing viewport tag can break mobile layout. A script loaded without `defer` at the top of `head` can slow the whole page down.

Try it

Save this as `index.html` and open it in your browser:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My first page</title>
  </head>
  <body>
```



```
<h1 id="welcome">Hello, web</h1>
<p class="lead">This is a paragraph of text.</p>
<!-- You can add more elements below. -->
</body>
</html>
```

Open DevTools and click the **Elements** tab. Expand the `<html>` node and look at what is inside. You should see the same **head** and **body** you wrote, with the elements nested inside them.

Try a few small changes and reload between each:

- Change the `<title>` and watch the browser tab update.
- Change the `<h1>` text and watch the page content update.
- Remove the `lang="en"` attribute. The page still works, but you have lost a small accessibility signal.
- Add a new attribute like `data-author="you"` to the `<h1>` . Notice that nothing visible changes, but the attribute is there in DevTools.
- Paste the file into an HTML validator and fix any mistakes it reports.

You now have a working file you can build on for the rest of this pack.

< 0 / 6 >

✓Knew 0 Items

★Learnt 0 Items

⏮Skipped 0 Items

↺ResetProgress

Test Your Knowledge

What is the difference between the head and the body?

[Click to Reveal the Answer](#)

✓Already Know that

★Didn't Know that

⏮Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Pricing Comparison Table

Next Lesson